

App Dev Project Report

Modern Application Development II (CS2006)

Arun Kumar M P
(23f3002829)

10th March 2026

Author

Name: Arun Kumar M P

Roll Number: 23f3002829

Student Email: 23f3002829@ds.study.iitm.ac.in

Brief Bio:

I am an undergraduate student pursuing a B.E. in Electronics and Communication Engineering. I am interested in web development, distributed systems, and data-driven applications. Through this project, I explored full-stack web development using Flask APIs, Vue.js, and asynchronous task processing with Celery and Redis.

AI / LLM Usage

AI tools such as ChatGPT & Copilot were used during development primarily for debugging errors, understanding framework concepts, and clarifying implementation approaches. The overall project architecture, integration of components, and implementation of features were performed by the student.

The approximate contribution of AI assistance in this project is estimated at **30–35%**, mainly in the form of conceptual guidance, templates, styling and troubleshooting rather than direct code generation.

Description

The objective of this project is to develop a **Placement Portal Application (PPA)** that enables institutes to manage campus recruitment activities efficiently.

The system supports three primary roles:

- **Admin**
 - Manages companies and placement drives
 - Approves or rejects company registrations
 - Views system statistics and student applications

- **Company**
 - Registers on the platform
 - Creates placement drives
 - Reviews student applications
 - Updates application status
- **Student**
 - Registers and maintains a profile
 - Views eligible placement drives
 - Applies for recruitment drives
 - Tracks application status and placement history

The application uses a **Vue.js frontend communicating with Flask REST APIs**, while **background jobs such as reminders and reports are handled asynchronously using Celery and Redis**.

Technologies Used

Technology	Purpose
Flask	Backend framework used to build REST APIs and handle server-side logic
Flask-RESTful	Provides structured API resources and routing for backend endpoints
SQLite	Lightweight relational database used for storing application data
SQLAlchemy	ORM used for defining database models and relationships
Vue.js	Frontend framework used to build a reactive user interface
Bootstrap	Used for styling and responsive UI components
Redis	Used as the message broker for Celery background tasks

Celery	Executes asynchronous tasks such as reminders, reports, and CSV exports
MailHog	Development tool used to capture and preview outgoing emails locally

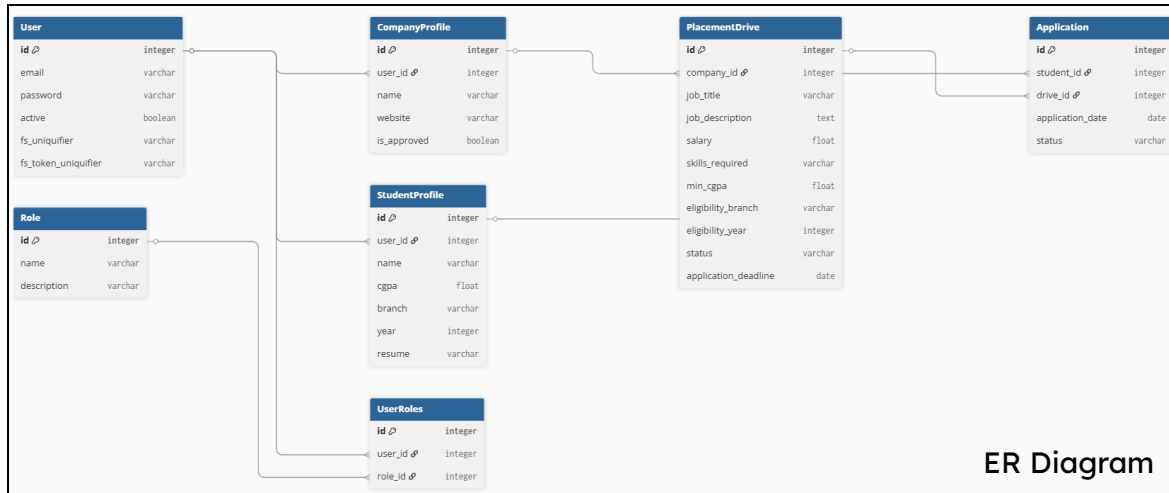
Database Schema Design

Table	Description	Key Fields
User	Stores authentication credentials and account status	id, email, password, active
Role	Defines system roles such as admin, company, and student	id, name, description
User Roles	Mapping table implementing many-to-many relationship between users and roles	user_id, role_id
Company Profile	Stores company information and approval status	id, user_id, name, website
Student Profile	Stores student academic and personal details	id, user_id, name, cgpa, branch, year
Placement Drive	Represents placement drives created by companies	id, company_id, job_title, salary, eligibility
Application	Stores student applications to placement drives	id, student_id, drive_id, status

Design Rationale

- Separation between **authentication entities (User, Role)** and **application entities (StudentProfile, CompanyProfile)**.

- Relationships ensure data consistency between students, companies, and placement drives.
- The schema supports role-based access control and scalable placement workflows.



API Design

The backend APIs are implemented using **Flask-RESTful resources**. Each API module is responsible for handling operations for a specific role or functionality.

API File	Purpose
auth_api.py	Handles authentication operations including login, registration, and token-based authentication
admin_api.py	Provides APIs for administrators to approve companies, manage placement drives, and view applications
company_api.py	Allows companies to create placement drives, view applicants, and update application status
student_api.py	Enables students to browse drives, apply for opportunities, and track application status
tasks_api.py	Provides endpoints to trigger Celery background jobs such as reminders and monthly reports

Architecture and Features

The project follows a **client-server architecture**.

Backend Structure

- Implemented using Flask
- Organized into modular controllers for each API group
- Uses SQLAlchemy models for database interaction
- Integrates Celery + Redis for background task execution

Frontend Structure

- Built using **Vue.js**
- Components organized into:
 - views
 - reusable components
 - router configuration
- Communicates with backend APIs via HTTP requests

Background Jobs Implemented

- **Daily Reminder Job**
 - Sends reminder emails to students about placement drives closing within the next two days
- **Monthly Activity Report**
 - Generates an HTML report summarizing placement activity and sends it to the administrator
- **CSV Export Job**
 - Generates a CSV file containing student application history

These tasks are executed asynchronously using Celery workers, ensuring that long-running operations do not block the main application.

Video

 Placement_Portal_App_23f3002829.mp4 (watch in 1.5x)
(<https://drive.google.com/file/d/15eKhj5Lwf3h-hMPByWPaZ3ggVG0pst0z/view?usp=sharing>)